


**UNIVERSIDADE FEDERAL DE SERGIPE**

**PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO**

**CLARIANE MARIA OLIVEIRA LISBOA**

**IRIS AZENATE FERREIRA**

**JOSÉ NILTON DOS SANTOS OLIVEIRA**

**LEONARDO SOUZA SILVA**

**THIAGO DE JESUS DOS SANTOS**

**VICTÓRIA CARDOSO DOS SANTOS**

# RECUPERAÇÃO EFICIENTE DE CONTEXTO EM ARTIGOS CIENTÍFICOS SOBRE MODELOS DE LINGUAGEM, RECUPERAÇÃO DE INFORMAÇÃO E RETRIEVAL-AUGMENTED GENERATION (RAG): SELEÇÃO DE CONTEXTO COM PROGRAMAÇÃO DINÂMICA

São Cristóvão, Sergipe
2026

**CLARIANE MARIA OLIVEIRA LISBOA**

**IRIS AZENATE FERREIRA**

**JOSÉ NILTON DOS SANTOS OLIVEIRA**

**LEONARDO SOUZA SILVA**

**THIAGO DE JESUS DOS SANTOS**

**VICTÓRIA CARDOSO DOS SANTOS**

# RECUPERAÇÃO EFICIENTE DE CONTEXTO EM ARTIGOS CIENTÍFICOS SOBRE MODELOS DE LINGUAGEM, RECUPERAÇÃO DE INFORMAÇÃO E RETRIEVAL-AUGMENTED GENERATION (RAG): SELEÇÃO DE CONTEXTO COM PROGRAMAÇÃO DINÂMICA

Trabalho apresentado à disciplina de Projeto e Análise de Algoritmos da Universidade Federal de Sergipe, como requisito parcial para obtenção de nota na referida disciplina.

Professor: Prof. Glauco Carneiro

São Cristóvão, Sergipe
2026

**SUMÁRIO**

**1 INTRODUÇÃO.....4**
**2 METODOLOGIA.....4**
2.1. Métodos de Seleção.....4
2.2. Ranking Simples.....5
2.3. Método Guloso.....5
2.4. Programação Dinâmica.....5
2.4.1. Implementação.....6
2.4.2. Reconstrução da Solução.....6
2.4.3. Demonstração da Programação Dinâmica.....7
2.5. Abordagem Exaustiva, Validação e Corretude.....8
**3. RESULTADOS.....8**
3.1. Caracterização dos custos e do orçamento de contexto.....8
**4 . ANÁLISE ASSINTÓTICA.....11**
**5. APLICAÇÃO NO RAG E LIMITAÇÕES.....12**
**6 . CONCLUSÃO.....12**
**7 DECLARAÇÃO DE USO DE IA GENERATIVA.....13**
**8 TABELA DE CONTRIBUIÇÃO INDIVIDUAL.....14**
**9 URL DO VÍDEO E REPOSITÓRIO.....15**
**REFERÊNCIAS BIBLIOGRÁFICAS.....15**

## 1 INTRODUÇÃO

A Geração Aumentada por Recuperação (Retrieval-Augmented Generation – RAG) combina modelos de linguagem com mecanismos de recuperação de informações externas para complementar o contexto utilizado na geração de respostas (GAO et al., 2023). Entretanto, a quantidade de documentos recuperados pode ultrapassar o orçamento disponível de tokens, tornando necessária a seleção dos conteúdos que serão enviados ao modelo.

Nesse contexto, a seleção de artigos científicos pode ser formulada como um problema da mochila 0/1, no qual cada documento possui um valor de relevância e um custo em tokens. O objetivo consiste em maximizar a soma das pontuações dos documentos selecionados sem ultrapassar a capacidade disponível. A programação dinâmica permite resolver essa formulação por meio da decomposição em subproblemas e do armazenamento de soluções intermediárias (MIT OPENCOURSEWARE, 2011; CP-ALGORITHMS, s.d.).

Este trabalho tem como objetivo avaliar a aplicação da programação dinâmica na seleção de artigos científicos recuperados da ACL Anthology, comparando seu desempenho com estratégias de ranking simples e seleção gulosa. São analisados a pontuação lexical dos documentos selecionados, o tempo de execução, o consumo de memória e as limitações computacionais dos métodos.

## 2 METODOLOGIA

O experimento deu continuidade à AV1, na qual a coleta foi configurada para até 18.000 artigos da ACL Anthology. Foram utilizados os primeiros 500 e 1.000 artigos, com seleção dos 20 ou 50 candidatos mais bem pontuados. Os custos foram calculados com o tokenizador “cl100k_base”, e os métodos Ranking AV1, Guloso e Programação Dinâmica foram comparados nos orçamentos de 708 e 1.416 tokens, considerando as consultas Q1, Q3 e Q5. Cada configuração foi executada três vezes, totalizando 216 medições. Foi executada no Google Colab, sistema: Linux-6.6.159+-x86_64-with-glibc2.3, python: 3.13.18. Foram utilizadas as bibliotecas NumPy 2.4.6, pandas 2.2.3, NLTK 3.10.3, sentence-transformers 5.7.0 e scikit-learn 1.6.1.

### 2.1. Métodos de Seleção

A seleção dos artigos foi formulada como um problema da mochila 0/1. Dados N artigos candidatos, cada um com custo $c_i$ em tokens e valor $v_i$ correspondente à pontuação lexical, busca-se maximizar a soma dos valores dos artigos selecionados, respeitando o

orçamento W. Foram implementados o Ranking AV1, o Guloso por razão valor/custo e a Programação Dinâmica, além da Busca Exaustiva para validação em instâncias pequenas.

As consultas Q1, Q3 e Q5 abordam recuperação em RAG, busca semântica e avaliação de sistemas RAG. Para avaliação da relevância, foram considerados como gabarito os 15 documentos com maior similaridade de cosseno calculada pelo modelo all-MiniLM-L6-v2. Foram utilizadas as métricas de precisão, Precision@5 e redundância por Jaccard. O consumo de memória foi medido com tracemalloc.

### 2.2. Ranking Simples

O Ranking AV1, desenvolvido na primeira avaliação da disciplina, foi reutilizado como estratégia de referência (baseline) para comparação com os demais métodos. O algoritmo ordena os artigos pela pontuação lexical em ordem decrescente e seleciona aqueles que cabem no orçamento disponível de tokens. Entretanto, por não considerar diretamente o custo dos documentos na ordenação, pode deixar de selecionar combinações com maior pontuação total. Sua implementação é apresentada no Quadro A.1, no Apêndice A.

### 2.3. Método Guloso

O método guloso foi implementado como estratégia de comparação, diferenciando-se do Ranking AV1 por ordenar os artigos pela razão entre a pontuação lexical e o custo em tokens, priorizando aqueles que oferecem maior relevância por token utilizado. A seleção ocorre sequencialmente, incluindo os artigos com pontuação positiva que cabem no orçamento restante. Em caso de empate na razão, prioriza-se a maior pontuação e, posteriormente, o identificador do artigo. Entretanto, por realizar escolhas locais sem reconsiderá-las, o método não garante a solução ótima para o problema da mochila 0/1. Sua implementação é apresentada no Quadro A.2, no Apêndice A.

### 2.4. Programação Dinâmica

A programação dinâmica é aplicável quando o problema apresenta duas propriedades (CORMEN et al., 2022): subestrutura ótima, isto é, a solução ótima contém soluções ótimas de subproblemas; e sobreposição de subproblemas, isto é, os mesmos subproblemas são requisitados repetidamente, o que justifica armazenar seus resultados (memoização ou tabulação) em vez de recalculá-los.

Define-se ***T[i][w]*** como o maior valor total obtenível utilizando apenas um subconjunto dos i primeiros artigos, com custo total menor ou igual a w. No código, ***T*** corresponde à matriz tabela. A resposta do problema original é ***T[N][W].***

A recorrência que define T é apresentada no Quadro 1.

**Quadro 1 –** Recorrência da programação dinâmica para a mochila 0/1

| | | |
|---|---|---|
| **Caso base:** | $T[0][w] = 0,$ | para todo $w$ em $\{0, \ldots, W\}$ |
| **Se $c_i > w$:** | $T[i][w] = T[i-1][w]$ | (o artigo $i$ não cabe) |
| **Se $c_i \leq w$:** | $T[i][w] = \max($ $T[i-1][w],$ | (não pegar o artigo $i$) |
| | $\quad T[i-1][w - c_i] + v_i)$ | (pegar o artigo $i$) |

Fonte: elaboração própria, com base em Cormen et al. (2022).

**Interpretação geométrica da tabela.** As linhas representam o número de artigos considerados *(de 0 a N)* e as colunas representam o orçamento de tokens *(de 0 a W)*. Por essa razão, a tabela possui dimensão *(N + 1) × (W + 1)*. A linha *i = 0* e a coluna *w = 0* existem para acomodar os casos-base.

2.4.1. Implementação

A implementação utiliza duas matrizes de mesma dimensão *(N + 1) × (W + 1)*, criadas com a biblioteca NumPy. A matriz tabela (tipo ponto flutuante) armazena os valores máximos ***T[i][w]***. Sua inicialização com zeros já satisfaz o caso base ***T[0][w] = 0***. A matriz escolhida (tipo booleano) é o histórico de decisão: *escolhido[i][w] = True* indica que, para o subproblema *(i, w)*, a melhor escolha foi incluir o artigo *i*; *False* indica que ele foi omitido. Por exemplo, *escolhido[2, 3] = True* significa que incluir o artigo 2 foi a melhor decisão para um orçamento de até 3 tokens.

O preenchimento é realizado linha a linha (artigo a artigo)**,** a instrução *tabela[i] = tabela[i - 1]* inicializa a linha corrente com a alternativa de não pegar o artigo. **Poda de artigos irrelevantes ou inviáveis**. Se *custo > W* (o artigo sozinho excede o orçamento total) ou *pontos == 0* (o artigo não contribui com relevância), o laço executa continue .**Vetorização por fatiamento**. Em vez de um laço interno sobre cada *w* de $c_i$ a *W*, o código compara todos os orçamentos de uma só vez. **Atualização das duas matrizes**. *np.where* seleciona, para cada orçamento, o maior dos dois valores, e escolhido recebe o resultado booleano da comparação, registrando a decisão tomada. A implementação completa é apresentada no Quadro A.3, no Apêndice A.

2.4.2. Reconstrução da Solução

Após o preenchimento, *tabela[N][W]* contém o valor ótimo, mas a composição da solução deve ser recuperada percorrendo-se a tabela de decisões do final para o início, conforme o Quadro A.3, no Apêndice A. O custo dessa etapa é *O(N)*. Duas asserções funcionam como verificações de integridade em tempo de execução: ***resultado['tokens'] <= W*** verifica a viabilidade: a soma dos custos dos artigos reconstruídos não excede o orçamento; ***math.isclose(resultado['valor']***, *tabela[quantidade, W], abs_tol=1e-7)* verifica a consistência entre a solução reconstruída e o valor ótimo calculado pela tabela

2.4.3. Demonstração da Programação Dinâmica

Considere uma mochila com orçamento W = 5 tokens e os artigos descritos na Tabela 1.

Tabela 1 - Instância de exemplo

| Artigo | Índice (*i*) | Custo (tokens) | Valor |
|---|---|---|---|
| A | 1 | 2 | 3 |
| B | 2 | 3 | 4 |
| C | 3 | 1 | 2 |

Fonte: elaboração própria (2026).

Como há 3 artigos e $W = 5$, as matrizes têm dimensão $(3 + 1) \times (5 + 1)$. A linha $i = 0$ constitui o caso base.

**Linha i = 1** *(artigo A: custo 2, valor 3)*; **Linha i = 2** *(artigo B: custo 3, valor 4)*; **Linha i = 3** *(artigo C: custo 1, valor 2)*. O resultado consolidado é apresentado nas Tabelas 2 e 3.

Tabela 2 - Matriz tabela (Resultados Máximos)

| | *w* = 0 | *w* = 1 | *w* = 2 | *w* = 3 | *w* = 4 | *w* = 5 |
|---|---|---|---|---|---|---|
| *i* = 0 (nenhum) | 0 | 0 | 0 | 0 | 0 | 0 |
| *i* = 1 (artigo A) | 0 | 0 | 3 | 3 | 3 | 3 |
| *i* = 2 (artigo B) | 0 | 0 | 3 | 4 | 4 | 7 |
| *i* = 3 (artigo C) | 0 | 2 | 3 | 5 | 6 | 7 |

Fonte: elaboração própria (2026).

Tabela 3 - Matriz escolhido (histórico de decisão)

| | $w = 0$ | $w = 1$ | $w = 2$ | $w = 3$ | $w = 4$ | $w = 5$ |
|---|---|---|---|---|---|---|
| $i = 1$ (artigo A) | False | False | True | True | True | True |
| $i = 2$ (artigo B) | False | False | False | True | True | True |
| $i = 3$ (artigo C) | False | True | False | True | True | False |

Fonte: elaboração própria (2026).

O valor ótimo é $T[3][5] = 7$. O resultado final é a combinação $A + B$, com valor máximo igual a 7 e custo total igual a 5 tokens, que esgota exatamente o orçamento.

### 2.5. Abordagem Exaustiva, Validação e Corretude

Para validar a programação dinâmica, implementou-se uma busca exaustiva, que enumera todos os $2^N$ subconjuntos possíveis e retém o melhor subconjunto viável, servindo como referência para instâncias pequenas. Sua implementação é apresentada no Quadro A.4, no Apêndice A.

A validação foi organizada em duas frentes complementares: (i) casos de teste dirigidos, que exploram situações de borda e entradas inválidas; e (ii) um teste diferencial de corretude, que confronta a programação dinâmica com a busca exaustiva em instâncias aleatórias. Os 12 testes realizados foram aprovados, incluindo 100% de concordância entre os métodos nas 30 instâncias aleatórias. Os resultados são apresentados no Quadro A.6, no Apêndice A. O código do teste diferencial é apresentado no Quadro A.5, no Apêndice A.

A corretude da recorrência fundamenta-se na subestrutura ótima: para cada artigo, a solução ótima considera sua inclusão ou exclusão. Por indução no número de artigos, o caso-base possui valor zero e cada etapa seleciona o maior valor entre as duas alternativas, preservando a solução ótima. O teste diferencial fornece evidência empírica de corretude em instâncias pequenas, mas não substitui a justificativa teórica. As evidências dos testes estão disponíveis no repositório do grupo, descrito na Seção 9. A validação das entradas rejeita orçamentos negativos, custos não positivos, valores inválidos e identificadores duplicados, enquanto os testes também contemplam conjuntos vazios, empates e artigos inviáveis.

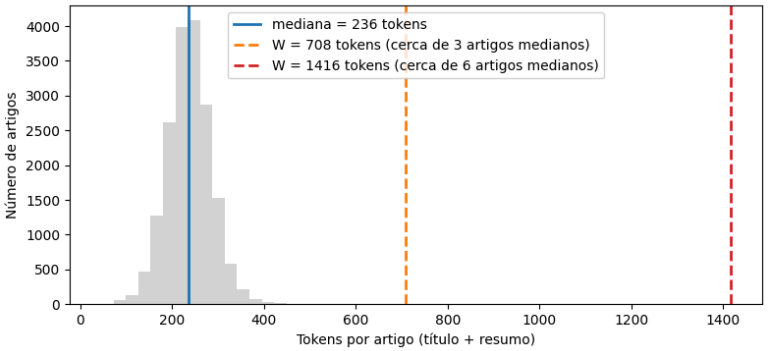
## 3. RESULTADOS

### 3.1. Caracterização dos custos e do orçamento de contexto

A distribuição dos custos dos artigos foi analisada para fundamentar a definição dos orçamentos experimentais. Na Figura 1, observa-se que a mediana dos 18.000 artigos foi de

236 tokens por artigo, utilizada como referência para estabelecer dois limites: 708 e 1.416 tokens, correspondentes a três e seis vezes a mediana, respectivamente.

Figura 1 — Posicionamento dos orçamentos experimentais em relação à distribuição dos custos dos artigos.




Fonte: elaboração própria (2026).

A Tabela 4 apresenta a comparação entre os métodos de seleção de contexto considerando o desempenho obtido e o custo computacional. A Programação Dinâmica (DP) alcançou os maiores scores nos dois tamanhos de corpus, com valor de 74,67. Entretanto, seu tempo de execução variou entre 1,2 e 1,3 ms, enquanto Ranking e Guloso apresentaram tempos entre 0,23 e 0,25 ms. O consumo de memória do DP atingiu aproximadamente 0,32 MB, em comparação com 0,0031 MB dos demais métodos. Em relação à relevância, o Ranking apresentou precisão de 0,722, a Precision@5 variou de 0,333 a 0,400 e a similaridade de Jaccard ficou entre 0,172 e 0,197.

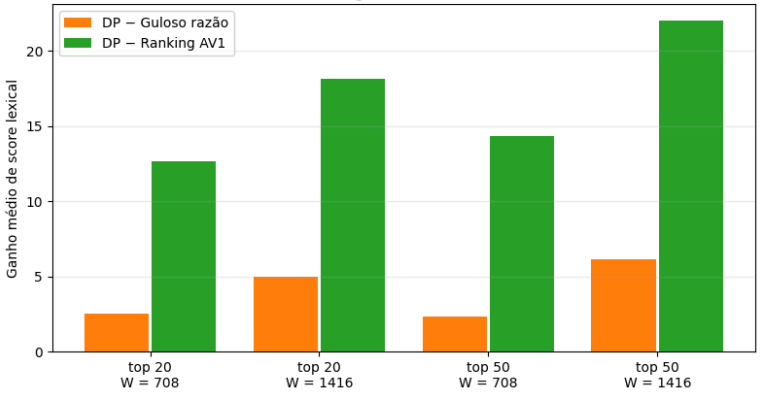
Tabela 4 – Comparação do desempenho e do custo computacional dos métodos de seleção de contexto. Valores médios para N = 50 candidatos e W = 708 tokens, considerando Q1, Q3 e Q5, com três repetições.

| Corpus | Método | Score | Tempo (ms) | Mem. (MB) | Tokens | Precisão | P@5 | Jaccard |
|---|---|---|---|---|---|---|---|---|
| 500 | Ranking | 60,33 | 0,235 | 0,0031 | 675,00 | 0,722 | 0,333 | 0,194 |
| 500 | Guloso | 72,67 | 0,246 | 0,0031 | 669,00 | 0,500 | 0,333 | 0,172 |
| 500 | DP | 74,67 | 1,318 | 0,3216 | 699,33 | 0,639 | 0,400 | 0,187 |
| 1.000 | Ranking | 60,33 | 0,231 | 0,0031 | 675,00 | 0,722 | 0,333 | 0,194 |
| 1.000 | Guloso | 72,00 | 0,240 | 0,0031 | 661,00 | 0,639 | 0,400 | 0,181 |
| 1.000 | DP | 74,67 | 1,196 | 0,3213 | 698,33 | 0,639 | 0,400 | 0,197 |

Fonte: elaboração própria (2026).

A Programação Dinâmica apresentou scores iguais ou superiores aos das estratégias de referência. No cenário de 500 artigos, 20 candidatos e orçamento de 708 tokens, o score médio foi de 73,00 para a Programação Dinâmica, 70,00 para o Guloso e 60,33 para o Ranking AV1, representando ganhos de aproximadamente 4,3% e 21,0%, respectivamente. Com 50 candidatos, o score da Programação Dinâmica aumentou para 74,67. A Programação Dinâmica superou o Guloso em 48 das 72 medições, correspondentes a 16 dos 24 cenários distintos. Nas demais comparações, os valores foram equivalentes.

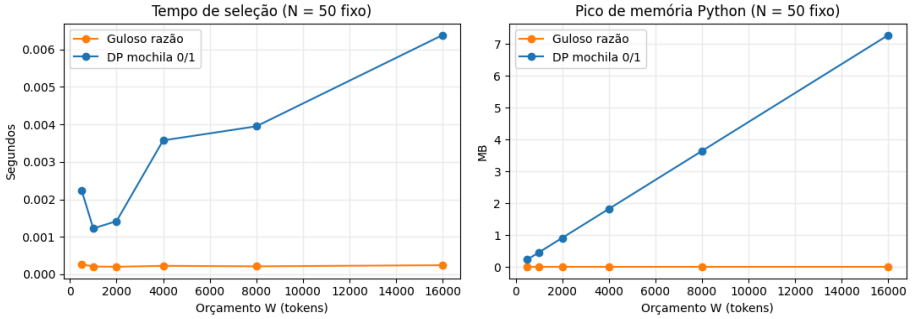
Figura 2 – Ganho médio de score lexical da Programação Dinâmica em relação ao algoritmo Guloso por razão e ao Ranking AV1 (diferença absoluta em pontos de score).




Fonte: elaboração própria (2026).

A Figura 3 apresenta o impacto do orçamento de tokens (W) sobre o tempo de seleção e o consumo de memória dos métodos Guloso e Programação Dinâmica, mantendo N = 50 candidatos. O Guloso apresentou tempo de execução relativamente estável e baixo consumo de memória à medida que o orçamento aumentou. Em contrapartida, a Programação Dinâmica apresentou maior custo computacional, com crescimento do tempo de execução e pico de memória de aproximadamente 7,3 MB para W = 16.000 tokens.

Figura 3 – Tempo de seleção e consumo de memória em função do orçamento de tokens para $N = 50$ candidatos.




Fonte: elaboração própria (2026).

O Ranking AV1 apresentou maior precisão (0,722), apesar do menor *score*, enquanto a Programação Dinâmica obteve precisão de 0,639. Esse resultado indica que maximizar a pontuação lexical não garante maior relevância segundo o gabarito semântico utilizado. Quanto à redundância, o Guloso apresentou os menores valores de Jaccard (0,172 a 0,181), enquanto a Programação Dinâmica apresentou valores entre 0,187 e 0,197. Assim, a otimização do "score" não implicou menor sobreposição entre os documentos selecionados.

## 4 . ANÁLISE ASSINTÓTICA

A análise assintótica considera o tempo de execução e o consumo de memória dos algoritmos utilizados na seleção de contexto. A Tabela 5 apresenta as complexidades do Ranking AV1, Guloso, Programação Dinâmica e Busca Exaustiva, considerando N como o número de artigos candidatos e W como o orçamento de tokens disponível.

Tabela 5 – Análise assintótica dos algoritmos de seleção de contexto.

| Algoritmo | Melhor caso | Pior caso | Complexidade espacial |
|---|---|---|---|
| Ranking AV1 | $\Theta(N \log N)$ | $\Theta(N \log N)$ | $\Theta(N)$ |
| Guloso | $\Theta(N \log N)$ | $\Theta(N \log N)$ | $\Theta(N)$ |
| Programação Dinâmica | $\Theta(N \times W)$ | $\Theta(N \times W)$ | $\Theta(N \times W)$ |
| Busca Exaustiva | $\Theta(N \times 2^N)$ | $\Theta(N \times 2^N)$ | $\Theta(N)$ |

Fonte: Elaborado pelos autores (2026), com base em Erickson (2019) e CP-Algorithms (2026).

Os métodos Ranking AV1 e Guloso apresentam complexidade temporal $\Theta(N \log N)$, devido à ordenação dos candidatos (Erickson, 2019), seguida de uma uníca passagem linear para preencher o orçamento. A complexidade espacial é $\Theta(N)$, referente à cópia ordenada da lista de candidatos. Ambos utilizam critérios de seleção que não garantem a solução ótima.

A Programação Dinâmica apresenta complexidade temporal e espacial $\Theta(N \times W)$, pois preenche uma tabela de $(N + 1) \times (W + 1)$ células, e cada célula é calculada em tempo constante a partir da linha anterior (CP-Algorithms, 2026; MIT OpenCourseWare, 2011). A reconstrução dos artigos selecionados percorre a tabela uma vez por item, com custo adicional $\Theta(N)$, que não altera a ordem de complexidade total. Já a Busca Exaustiva possui custo temporal exponencial, $\Theta(N \times 2^N)$, pois avalia os $2^N$ subconjuntos possíveis e soma os custos e valores de cada um em tempo $O(N)$. Por esse motivo, ela é utilizada apenas para

validar a solução da Programação Dinâmica em instâncias pequenas, com no máximo 20 itens.

A complexidade da Programação Dinâmica é considerada pseudopolinomial, pois depende do valor numérico de W, e não apenas do tamanho de sua representação na entrada, que ocupa O(log W) bits (Erickson, 2019). Esse comportamento é compatível com os resultados experimentais: as matrizes “tabela” e “escolhido” utilizam aproximadamente 9(N + 1)(W + 1) bytes, considerando os tipos “float64” e booleano. Para N = 50, os picos de memória medidos foram de aproximadamente 0,32 MB para W = 708 e 7,3 MB para W = 16.000, confirmando o crescimento do consumo de memória com o orçamento disponível.

O modelo RAM considera apenas a etapa de seleção. Ficam fora dele a tokenização e a construção do índice, executadas na preparação dos dados, a geração dos embeddings do gabarito, utilizada apenas na avaliação, o acesso a disco e, em uma aplicação RAG real, a geração de respostas pelo LLM, cujo custo não foi mensurado neste experimento.

## 5. APLICAÇÃO NO RAG E LIMITAÇÕES

Em sistemas de Geração Aumentada por Recuperação (Retrieval-Augmented Generation – RAG), a Programação Dinâmica pode ser utilizada para selecionar artigos relevantes que comporão o contexto enviado a um modelo de linguagem, respeitando o orçamento de tokens. No exemplo implementado, considerando Q1, 500 artigos, N = 20 e W = 708, foram selecionados três artigos, totalizando 706 tokens, com identificação da fonte e URL de cada documento.

Entretanto, a abordagem apresenta limitações, como o aumento do consumo de memória em orçamentos elevados e a dependência de pontuações lexicais, que não garantem relevância semântica. Além disso, o orçamento considera apenas os textos dos artigos, o gabarito semântico é uma aproximação automática e o experimento não avaliou respostas geradas por um LLM. Portanto, não é possível afirmar que a otimização da seleção melhora diretamente a qualidade das respostas em sistemas RAG.

## 6 . CONCLUSÃO

Os resultados indicam que a Programação Dinâmica é eficaz para maximizar a soma dos scores lexicais dos documentos selecionados sob um orçamento fixo de tokens, encontrando a combinação ótima para o modelo de seleção adotado. A abordagem obteve scores iguais ou superiores aos das estratégias de referência nas configurações avaliadas.

Entretanto, o método Guloso apresentou menor custo computacional, embora não garanta a solução ótima.

Além disso, o Ranking AV1 obteve maior precisão semântica (0,722) que a Programação Dinâmica (0,639), indicando que maximizar o score lexical não garante a seleção dos documentos mais relevantes segundo o gabarito utilizado. Dessa forma, a escolha do algoritmo envolve um equilíbrio entre a qualidade da seleção e o custo computacional, considerando as restrições e os objetivos da aplicação. Ressalta-se que a maior pontuação lexical não garante, por si só, melhor qualidade das respostas geradas por modelos de linguagem.

## 7 DECLARAÇÃO DE USO DE IA GENERATIVA

Em conformidade com as orientações de transparência no uso de Inteligência Artificial Generativa, os autores declaram a utilização da ferramenta ChatGPT, desenvolvida pela OpenAI, como recurso de apoio à revisão textual, à organização das informações e à conferência de consistência técnica do presente trabalho. Durante a revisão do relatório, foi utilizado o modelo GPT-6.

A ferramenta foi empregada de forma complementar, auxiliando na identificação de possíveis inconsistências, na melhoria da clareza da linguagem técnica e na organização da apresentação dos métodos e resultados. As sugestões foram avaliadas criticamente pelos integrantes, considerando as implementações e os dados obtidos no experimento.

Quadro 2 – Registro de utilização da IA generativa.

| **Prompt utilizado (resumido)** | **Tratamento da sugestão** |
|---|---|
| “Revise a clareza e a organização da descrição dos métodos de seleção.” | **Aproveitada parcialmente:** ajustes na redação e na organização das subseções, preservando a metodologia apresentada pelo grupo. |
| “Verifique se a descrição da Programação Dinâmica está coerente com o código implementado.” | **Aproveitada após conferência:** revisão da explicação das matrizes, do preenchimento e da reconstrução da solução, com base no notebook. |
| “Confira a consistência entre as métricas, as tabelas e a discussão dos resultados.” | **Corrigida após verificação:** ajustes pontuais na apresentação das medições e na interpretação dos resultados experimentais. |

| | |
|---|---|
| “Sugira formas de reduzir o relatório sem comprometer as informações essenciais.” | **Parcialmente rejeitada:** a retirada de figuras não foi adotada; foram mantidos os gráficos e reduzidos apenas trechos repetitivos. |
| “Verifique se a implementação da Programação Dinâmica para o problema da mochila 0/1 está correta, considerando o preenchimento da tabela, a reconstrução da solução e o respeito ao orçamento de tokens.” | **Aproveitada como apoio à revisão**: as observações foram conferidas no código e a corretude foi avaliada por meio de testes automatizados, incluindo a comparação com a Busca Exaustiva em 30 instâncias aleatórias. |

**Fonte:** Elaborado pelos autores (2026).

Os autores permanecem responsáveis pelas decisões metodológicas, pela validação das implementações, pela interpretação dos resultados e pela integridade do conteúdo apresentado. A ferramenta de IA generativa não foi incluída como autora do trabalho.

## 8 TABELA DE CONTRIBUIÇÃO INDIVIDUAL

Quadro 6 - Contribuição individual dos integrantes.

| Integrante | Responsabilidade principal | Contribuições específicas |
|---|---|---|
| Clariane Maria Oliveira Lisboa | Modelagem do problema da mochila 0/1 | Definição dos orçamentos de tokens, implementação da validação das entradas e padronização do formato de saída dos algoritmos. Edição do vídeo. |
| Iris Azenate Ferreira | Preparação do corpus e configuração do experimento | Configuração do ambiente, obtenção e preparação dos artigos da ACL Anthology, contagem de tokens e análise da distribuição dos custos. |
| José Nilton dos Santos Oliveira | Implementação dos métodos de referência | Reutilização da busca e do índice invertido da AV1, geração dos candidatos e implementação do Ranking AV1 e do método Guloso. |
| Leonardo Souza Silva | Implementação e validação da programação dinâmica | Desenvolvimento da programação dinâmica e da busca exaustiva, implementação dos testes funcionais e de integridade e análise de corretude. |
| Thiago de Jesus dos Santos | Planejamento experimental e análise dos resultados | Definição e execução das configurações experimentais, coleta das métricas de desempenho e relevância, elaboração das tabelas e gráficos comparativos. |

| | | |
|---|---|---|
| Victória Cardoso dos Santos | Análise assintótica e aplicação no RAG | Análise das complexidades temporal e espacial, avaliação do comportamento pseudopolinomial, montagem do contexto para RAG, discussão das limitações e registro de informações de reprodutibilidade. |

Fonte: Elaborado pelos autores.

Além das responsabilidades descritas, os integrantes também colaboraram na revisão geral do trabalho, na preparação da apresentação e na gravação do vídeo. A organização e a disponibilização dos arquivos no repositório ficaram sob responsabilidade de Leonardo Souza Silva.

## 9 URL DO VÍDEO E REPOSITÓRIO

Repositório Github:
https://github.com/Leonardodcomp/PAA_UFS_2026_2_Ferreira_Iris_Ferreira_Jose_Lisboa_Clariane_Santos_Thiago_Santos_Victoria_Silva_Leo/tree/main/AV02

URL do vídeo:
https://drive.google.com/file/d/1MBIx6AbKiQ3P_lAueVdiaLOKJwhNf9he/view

## REFERÊNCIAS BIBLIOGRÁFICAS

ASSOCIATION FOR COMPUTATIONAL LINGUISTICS. **ACL Anthology**. [S. l.]: Association for Computational Linguistics, [s. d.]. Disponível em: https://aclanthology.org/. Acesso em: 23 set. 2026.

CP-ALGORITHMS. **Knapsack Problem.** [S. l.], [s. d.]. Disponível em: https://cp-algorithms.com/dynamic_programming/knapsack.html.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Introduction to Algorithms**. 4. ed. Cambridge, MA: MIT Press, 2022.

ERICKSON, Jeff. **Algorithms**. Urbana: [s.n.], 2019. Disponível em: https://jeffe.cs.illinois.edu/teaching/algorithms/. Acesso em: 15 set. 2026.

GAO, Yunfan et al. **Retrieval-Augmented Generation for Large Language Models: A Surve**y. arXiv, 2023. Disponível em: https://arxiv.org/abs/2312.10997.

MIT OPENCOURSEWARE. **Dynamic Programming: Knapsack Problem**. Cambridge: Massachusetts Institute of Technology, 2011. Disponível em: https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/.

SENTENCE TRANSFORMERS. **all-MiniLM-L6-v2.** [S.l.]: Hugging Face, [2021?]. Disponível em: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Acesso em: 20 set. 2026.

# APÊNDICE A – IMPLEMENTAÇÕES DOS ALGORITMOS

**Quadro A.1 – Implementação do ranking simples.**

```python
def ranking_av1(itens, W):
    """Baseline: tenta os artigos com maiores pontuações primeiro."""
    validar_entrada(itens, W)
    ordem = sorted(itens, key=lambda a: (-a['value'], a['id']))
    escolhidos = []
    espaco_livre = W
    for artigo in ordem:
        if artigo['value'] > 0 and artigo['cost'] <= espaco_livre:
            escolhidos.append(artigo)
            espaco_livre -= artigo['cost']
    return resposta(escolhidos)
```

Fonte: elaboração própria (2026).

**Quadro A.2 – Implementação do método guloso por razão valor/custo**

```python
def guloso_razao(itens, W):
    """Guloso: prefere maior pontuação dividida pelo custo."""
    validar_entrada(itens, W)
    ordem = sorted(itens, key=lambda a: (-a['value']/a['cost'],
-a['value'], a['id']))
    escolhidos = []
    espaco_livre = W
    for artigo in ordem:
        if artigo['value'] > 0 and artigo['cost'] <= espaco_livre:
            escolhidos.append(artigo)
            espaco_livre -= artigo['cost']
    return resposta(escolhidos)
```

Fonte: elaboração própria (2026).

**Quadro A.3 – Implementação da Programação Dinâmica (mochila 0/1)**

```python
def mochila_dp(itens, W):
```

```
    """Programação dinâmica: encontra a melhor soma de pontuações."""
    validar_entrada(itens, W)
    quantidade = len(itens)

    # Tabela de valores e tabela para lembrar quais artigos foram
escolhidos.
    tabela = np.zeros((quantidade + 1, W + 1))
    escolhido = np.zeros((quantidade + 1, W + 1), dtype=bool)

    for i, artigo in enumerate(itens, start=1):
        custo = int(artigo['cost'])
        pontos = float(artigo['value'])

        # Começa com a opção de NÃO pegar o artigo atual.
        tabela[i] = tabela[i - 1]
        if custo > W or pontos == 0:
            continue

        # Se pegar o artigo, o valor é o anterior + pontos dele.
        # O fatiamento faz a comparação para todos os orçamentos de uma
vez.
        valor_se_pegar = tabela[i - 1, :W + 1 - custo] + pontos
        valor_sem_pegar = tabela[i, custo:]
        vale_a_pena = valor_se_pegar > valor_sem_pegar + 1e-10

        tabela[i, custo:] = np.where(vale_a_pena, valor_se_pegar,
valor_sem_pegar)
        escolhido[i, custo:] = vale_a_pena

    # Caminha do final da tabela até o início para descobrir os IDs.
    espaco = W
    artigos_escolhidos = []
    for i in range(quantidade, 0, -1):
        if escolhido[i, espaco]:
            artigo = itens[i - 1]
            artigos_escolhidos.append(artigo)
            espaco -= int(artigo['cost'])

    artigos_escolhidos.reverse()
    resultado = resposta(artigos_escolhidos)
    assert resultado['tokens'] <= W
    assert math.isclose(
        resultado['valor'],
```

```
        tabela[quantidade, W],
        abs_tol=1e-7
    )
    return resultado
```

Fonte: elaboração própria (2026).

**Quadro A.4 – Implementação da busca exaustiva.**

Python

```
def exaustiva(itens, W):
    """Confere todas as combinações (somente para testes pequenos)."""
    validar_entrada(itens, W)
    if len(itens) > 20:
        raise ValueError('Use no máximo 20 artigos na busca exaustiva')

    melhor = resposta([])
    for mascara in range(2 ** len(itens)):
        escolhidos = [a for i, a in enumerate(itens) if mascara & (1 <<
i)]
        atual = resposta(escolhidos)
        if atual['tokens'] <= W and atual['valor'] > melhor['valor'] +
1e-10:
            melhor = atual
    return melhor
```

Fonte: elaboração própria (2026).

**Quadro A5 –Teste Diferencial de Corretude.**

Python

```
# Comparação com exaustiva em instâncias pequenas aleatórias
rng=random.Random(SEED)
ok=True
for rodada in range(30):
    xs=[item(f'{rodada}-{j}',rng.randint(1,12),rng.randint(0,15)) for j
in range(rng.randint(0,12))]
    w=rng.randint(0,40)
    if not
math.isclose(mochila_dp(xs,w)['valor'],exaustiva(xs,w)['valor'],abs_tol
=1e-7):ok=False;break
```

```
testes.append({'teste':'12. DP × exaustiva (30
instâncias)','esperado':'mesmo valor ótimo nas 30',
               'observado':'30 concordâncias' if ok else f'divergência
na rodada {rodada}',
               'status':'PASSOU' if ok else
'FALHOU','requisito':'corretude'})
```

Fonte: elaboração própria (2026).

**Quadro A6 –Teste Diferencial de Corretude.**

| Nº | Caso de teste | Cenário | O que se verifica |
|---|---|---|---|
| 1 | Uso máximo de tokens (teste padrão) | Instância típica em que o orçamento pode ser preenchido | Valor ótimo e respeito ao limite $W$ na seleção retornada |
| 2 | Corpus vazio | $N = 0$ | Tabela de dimensão $1 \times (W + 1)$ com zeros; seleção vazia e valor 0, sem erro de execução |
| 3 | Tokens excedidos | Orçamento zerado ($W = 0$) | Nenhum artigo com custo positivo pode ser selecionado; resultado vazio |
| 4 | Borda exata | Artigo cujo custo é igual ao orçamento | Ausência de erro de índice (*off-by-one*) no fatiamento `custo:` e aceitação de custo = $w$ (condição $\leq$, não $<$) |
| 5 | Artigo inviável | Custo do artigo superior ao orçamento ($c_i > W$) | Desvio por `continue`; o artigo nunca é selecionado |
| 6 | Tratamento de empates | Dois artigos com mesmo custo e mesmo valor | Resultado determinístico segundo o critério estrito com tolerância |
| 7 | Artigo sem relevância | Artigo com valor = 0 | Artigo ignorado; nenhum token é gasto sem ganho |
| 8 | Alto orçamento de tokens | $W$ elevado, seleção de grande quantidade de artigos | Escalabilidade: o tempo cresce conforme $O(N \cdot W)$, sem degradação anormal, graças à vetorização |
| 9 | Caso adversarial simples | Instância em que a estratégia gulosa falha (como a da Seção 5) | Refutação do método guloso: a programação dinâmica atinge o ótimo (7) e o guloso, não (5) |
| 10 | Consulta sem correspondência | Nenhum artigo com relevância para a consulta | Todos os valores nulos; seleção vazia e valor 0 |
| 11 | Rejeição de IDs duplicados | Entrada com identificadores repetidos | `validar_entrada` rejeita a entrada, preservando a integridade da reconstrução, que identifica por ID |

Fonte: elaboração própria (2026).